DOI:10.1145/3009824

Article development led by acmqueue
queue.acm.org

Measuring bottleneck bandwidth and round-trip propagation time.

BY NEAL CARDWELL, YUCHUNG CHENG, C. STEPHEN GUNN, SOHEIL HASSAS YEGANEH, AND VAN JACOBSON

BBR: Congestion-Based Congestion Control

BY ALL ACCOUNTS, today's Internet is not moving data as well as it should. Most of the world's cellular users experience delays of seconds to minutes; public Wi-Fi in airports and conference venues is often worse. Physics and climate researchers need to exchange petabytes of data with global collaborators but find their carefully engineered multi-Gbps infrastructure often delivers at only a few Mbps over intercontinental distances.[6]

These problems result from a design choice made when TCP congestion control was created in the 1980s—interpreting packet loss as "congestion."[13] This equivalence was true at the time but was because of technology limitations, not first principles. As NICs (network interface controllers) evolved from Mbps to Gbps and memory chips from KB to GB, the relationship between packet loss and congestion became more tenuous.

Today TCP's loss-based congestion control—even with the current best of breed, CUBIC[11]—is the primary cause of these problems. When bottleneck buffers are large, loss-based congestion control keeps them full, causing bufferbloat. When bottleneck buffers are small, loss-based congestion control misinterprets loss as a signal of congestion, leading to low throughput. Fixing these problems requires an alternative to loss-based congestion control. Finding this alternative requires an understanding of where and how network congestion originates.

Congestion and Bottlenecks

At any time, a (full-duplex) TCP connection has exactly one slowest link or *bottleneck* in each direction. The bottleneck is important because:

- It determines the connection's maximum data-delivery rate. This is a general property of incompressible flow (for example, picture a six-lane freeway at rush hour where an accident has reduced one short section to a single lane. The traffic upstream of the accident moves no faster than the traffic through that lane).
- It is where persistent queues form. Queues shrink only when a link's departure rate exceeds its arrival rate. For a connection running at maximum delivery rate, all links upstream of the bottleneck have a faster departure rate so their queues migrate to the bottleneck.

Regardless of how many links a connection traverses or what their individual speeds are, from TCP's viewpoint an arbitrarily complex path behaves as a single link with the same RTT (round-trip time) and bottleneck rate. Two physical constraints, *RTprop* (round-trip propagation time) and *BtlBw* (bottleneck bandwidth), bound transport performance. (If the network path were a physical pipe, *RTprop* would be its length and *BtlBw* its minimum diameter.)

Figure 1 shows RTT and delivery rate variation with the amount of data *in flight* (data sent but not yet acknowledged). Blue lines show the *RTprop* constraint, green lines the *BtlBw* constraint, and red lines the bottleneck buffer. Operation in the shaded regions is not possible since it would vio-

IMAGE FROM SHUTTERSTOCK.COM

late at least one constraint. Transitions between constraints result in three different regions (app-limited, bandwidth-limited, and buffer-limited) with qualitatively different behavior.

When there isn't enough data in flight to fill the pipe, *RTprop* determines behavior; otherwise, *BtlBw* dominates. Constraint lines intersect at *inflight* = *BtlBw* × *RTprop*, a.k.a. the pipe's BDP (bandwidth-delay product). Since the pipe is full past this point, the *inflight–BDP* excess creates a queue at the bottleneck, which results in the linear dependence of RTT on *inflight* data shown in the upper graph. Packets are dropped when the excess exceeds the buffer capacity. *Congestion* is just sustained operation to the right of the BDP line, and *congestion control* is some scheme to bound how far to the right a connection operates on average.

Loss-based congestion control operates at the right edge of the bandwidth-limited region, delivering full bottleneck bandwidth at the cost of high delay and frequent packet loss. When memory was expensive buffer sizes were only slightly larger than the BDP, which minimized loss-based congestion control's excess delay. Subsequent memory price decreases resulted in buffers orders of magnitude larger than ISP link BDPs, and the resulting bufferbloat yielded RTTs of seconds instead of milliseconds.[9]

The left edge of the bandwidth-limited region is a better operating point than the right. In 1979, Leonard Klein-

rock[16] showed this operating point was optimal, maximizing delivered bandwidth while minimizing delay and loss, both for individual connections and for the network as a whole[8]. Unfortunately, around the same time Jeffrey M. Jaffe[14] proved it was impossible to create a distributed algorithm that converged to this operating point. This result changed the direction of research from finding a distributed algorithm that achieved Kleinrock's optimal operating point to investigating different approaches to congestion control.

Our group at Google spends hours each day examining TCP packet header captures from all over the world, making sense of behavior anomalies and pathologies. Our usual first step is finding the essential path characteristics, *RTprop* and *BtlBw*. That these can be inferred from traces suggests that Jaffe's result might not be as limiting as it once appeared. His result rests on fundamental measurement ambiguities (For example, whether a measured RTT increase is caused by a path-length change, bottleneck bandwidth decrease, or queuing delay increase from another connection's traffic). Although it is impossible to disambiguate any single measurement, a connection's behavior over time tells a clearer story, suggesting the possibility of measurement strategies designed to resolve ambiguity.

Combining these measurements with a robust servo loop using recent control systems advances[12] could result in a distributed congestion-control protocol that reacts to actual congestion, not packet loss or transient queue delay, and converges with high probability to Kleinrock's optimal operating point. Thus began our three-year quest to create a congestion control based on measuring the two parameters that characterize a path: bottleneck bandwidth and round-trip propagation time, or BBR.

Figure 1. Delivery rate and round-trip time vs. inflight.

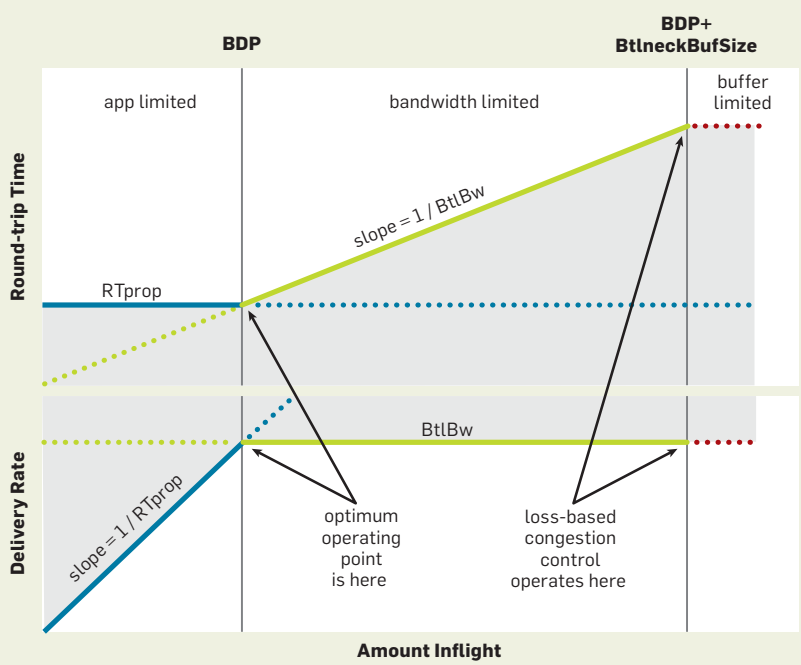


Figure 2. Ack-arrival half of the BBR algorithm.

```
function onAck(packet)
  rtt = now - packet.sendtime
  update_min_filter(RTpropFilter, rtt)
  delivered += packet.size
  delivered_time = now
  deliveryRate = (delivered - packet.delivered) /
                 (delivered_time - packet.delivered_time)
  if (deliveryRate > BtlBwFilter.currentMax
      || ! packet.app_limited)
      update_max_filter(BtlBwFilter, deliveryRate)
  if (app_limited_until > 0)
      app_limited_until = app_limited_until - packet.size
```

Characterizing the Bottleneck

A connection runs with the highest throughput and lowest delay when (rate balance) the bottleneck packet arrival rate equals *BtlBw* and (full pipe) the total data in flight is equal to the BDP (= *BtlBw* × *RTprop*).

The first condition guarantees that the bottleneck can run at 100% utilization. The second guarantees there is enough data to prevent bottleneck starvation but not overfill the pipe. The rate balance condition alone *does not* ensure there is no queue, only that it cannot change size (for example, if a connection starts by sending its 10-packet Initial Window into a five-packet BDP, then runs at exactly the bottleneck rate, five of the 10 initial packets fill the pipe so the excess forms a standing queue at the bottleneck that cannot dissipate). Similarly, the full pipe condition does not guarantee there is no queue (for example, a connection sending a BDP in BDP/2 bursts gets full bottleneck utilization, but with an average queue of BDP/4). The only way to minimize the queue at the bottleneck and all along the path is to meet both conditions simultaneously.

BtlBw and *RTprop* vary over the life of a connection, so they must be continuously estimated. TCP currently tracks RTT (the time interval from sending a data packet until it is acknowledged) since it is required for loss detection. At any time t,

$$RTT_t = RTprop_t + \eta_t$$

where $\eta \geq 0$ represents the "noise" introduced by queues along the path, the receiver's delayed ack strategy, ack aggregation, and so on. *RTprop* is a physical property of the connection's path and changes only when the path changes. Since path changes happen on time scales » *RTprop*, an unbiased, efficient estimator at time T is

$$\widehat{RTprop} = RTprop + \min(\eta_t) = \min(RTT_t)\ \forall t \in [T - W_R, T]$$

(That is, a running min over time window W_R (which is typically tens of seconds to minutes).

Unlike RTT, nothing in the TCP spec requires implementations to track bottleneck bandwidth, but a good estimate results from tracking delivery rate. When the ack for some packet arrives back at the sender, it conveys that packet's RTT and announces the delivery of data *inflight* when that packet departed. Average delivery rate between send and ack is the ratio of data delivered to time elapsed: `deliveryRate` $= \Delta delivered/\Delta t$. This rate must be $\leq$ the bottleneck rate (the arrival amount is known exactly so all the uncertainty is in the Δt, which must be $\geq$ the true arrival interval; thus, the ratio must be $\leq$ the true delivery rate, which is, in turn, upper-bounded by the bottleneck capacity). Therefore, a windowed-max of delivery rate is an efficient, unbiased estimator of *BtlBw*:

$$\widehat{BtlBw} = \max(\texttt{deliveryRate}_t)\ \forall t \in [T - W_B, T]$$

where the time window W_B is typically six to 10 RTTs.

TCP must record the departure time of each packet to compute RTT. BBR augments that record with the total data delivered so each ack arrival yields both an RTT and a delivery rate measurement that the filters convert to *RTprop* and *BtlBw* estimates.

Note that these values are completely independent: *RTprop* can change (for example, on a route change) but still have the same bottleneck, or *BtlBw* can change (for example, when a wireless link changes rate) without the path changing. (This independence is why both constraints have to be known to match sending behavior to delivery path.) Since *RTprop* is visible only to the left of BDP and *BtlBw* only to the right in Figure 1, they obey an uncertainty principle: whenever one can be measured, the other cannot. Intuitively, this is because the pipe has to be overfilled to find its capacity, which creates a queue that obscures the length of the pipe. For example, an application running a request/response protocol might never send enough data to fill the pipe and observe *BtlBw*. A multi-hour bulk data transfer might spend its entire lifetime in the bandwidth-limited region and have only a single sample of *RTprop* from the first packet's RTT. This intrinsic uncertainty means that in addition to estimators to recover the two path parameters, there must be states that track both what can be learned at the current operating point and, as information becomes stale, how to get to an operating point where it can be relearned.

Our group at Google spends hours each day examining TCP packet header captures from all over the world, making sense of behavior anomolies and pathologies. Our usual first step is finding the essential path characteristics, *RTprop* and *BtlBw*.

Matching the Packet Flow to the Delivery Path

The core BBR algorithm has two parts:

When an ack is received. Each ack provides new RTT and delivery rate measurements that update the *RTprop* and *BtlBw* estimates, as illustrated in Figure 2.

The `if` checks address the uncertainty issue described in the last paragraph: senders can be application limited, meaning the application runs out of data to fill the network. This is quite common because of request/response traffic. When there is a send opportunity but no data to send, BBR marks the corresponding bandwidth sample(s) as application limited (see `send()` pseudocode to follow). The code here decides which samples to include in the bandwidth model so it reflects network, not application, limits. *BtlBw* is a hard upper bound on the delivery rate so a measured delivery rate larger than the current *BtlBw* estimate must mean the estimate is too low, whether or not the sample was app-limited. Otherwise, application-limited samples are discarded. (Figure 1 shows that in the app-limited region `deliveryRate` underestimates, *BtlBw*. These checks prevent filling the *BtlBw* filter with underestimates that would cause data to be sent too slowly.)

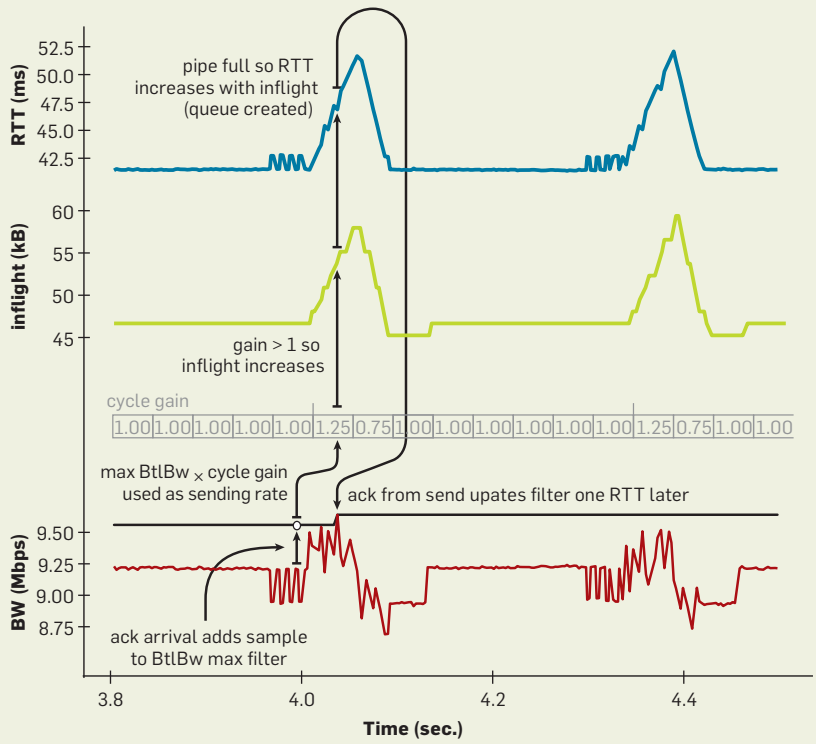
When data is sent. To match the packet-arrival rate to the bottleneck link's departure rate, BBR paces every data packet. BBR must match the bottleneck *rate*, which means pacing is integral to the design and fundamental to operation—pacing_rate is BBR's primary control parameter. A secondary parameter, cwnd_gain, bounds *inflight* to a small multiple of the BDP to handle common network and receiver pathologies (as we will discuss). Conceptually, the TCP send routine looks like

Figure 3. Packet-send half of the BBR algorithm.

```
function send(packet)
    bdp = BtlBwFilter.currentMax
          * RTpropFilter.currentMin
    if (inflight >= cwnd_gain * bdp)
        // wait for ack or retransmission timeout
        return
    if (now >= nextSendTime)
        packet = nextPacketToSend()
        if (! packet)
            app_limited_until = inflight
            return
        packet.app_limited = (app_limited_until > 0)
        packet.sendtime = now
        packet.delivered = delivered
        packet.delivered_time = delivered_time
        ship(packet)
        nextSendTime = now + packet.size /
               (pacing_gain * BtlBwFilter.currentMax)

    timerCallbackAt(send, nextSendTime)
```

Figure 4. RTT (blue), inflight (green), and delivery rate (red) detail.



the code in Figure 3. (In Linux, sending uses the efficient FQ/pacing queuing discipline,[4] which gives BBR line-rate single-connection performance on multigigabit links and handles thousands of lower-rate paced connections with negligible CPU overhead.)

Steady-state behavior. The rate and amount BBR sends is solely a function of the estimated *BtlBw* and *RTprop*, so the filters control adaptation in addition to estimating the bottleneck constraints. This creates the novel control loop shown in Figure 4, which illustrates the RTT (blue), *inflight* (green) and delivery rate (red) detail from 700ms of a 10Mbps, 40ms flow. The thick gray line above the delivery-rate data is the state of the *BtlBw* max filter. The triangular structures result from BBR cycling pacing_gain to determine if *BtlBw* has increased. The gain used for each part of the cycle is shown time-aligned with the data it influenced. The gain is applied an RTT earlier, when the data is sent. This is indicated by the horizontal jog in the event sequence description running up the left side.

BBR minimizes delay by spending most of its time with one BDP in flight, paced at the *BtlBw* estimate. This moves the bottleneck to the sender so it cannot observe *BtlBw* increases. Consequently, BBR periodically spends an *RTprop* interval at a pacing_gain > 1, which increases the sending rate and *inflight*. If *BtlBw* hasn't changed, then a queue is created at the bottleneck, increasing RTT, which keeps `deliveryRate` constant. (This queue is removed by sending at a compensating pacing_gain < 1 for the next *RTprop*.) If *BtlBw* has increased, `deliveryRate` increases and the new max immediately increases the *BtlBw* filter output, increasing the base pacing rate. Thus, BBR converges to the new bottleneck rate exponentially fast. Figure 5 shows the effect on a 10Mbps, 40ms flow of *BtlBw* abruptly doubling to 20Mbps after 20 seconds of steady operation (top graph) then dropping to 10Mbps after another 20 seconds of steady operation at 20Mbps (bottom graph).

(BBR is a simple instance of a Max-plus control system, a new approach to control based on nonstandard algebra.[12] This approach allows the adaptation rate [controlled by the *max* gain] to be independent of the queue growth [controlled by the *average* gain]. Applied to this problem, it results in a simple, implicit control loop where the adaptation to physical constraint changes is automatically handled by the filters representing those constraints. A conventional control system would require multiple loops connected by a complex state machine to accomplish the same result.)

Single BBR Flow Startup Behavior

Existing implementations handle events such as startup, shutdown, and loss recovery with event-specific algorithms and many lines of code. BBR uses the code detailed earlier for everything, handling events by sequencing through a set of "states" that are defined by a table containing one or more fixed gains and exit criteria. Most of the time is spent in the ProbeBW state described in the section on Steady-state Behavior.

The Startup and Drain states are used at connection start (Figure 6). To handle Internet link bandwidths spanning 12 orders of magnitude, Startup implements a binary search for *BtlBw* by using a gain of 2/ln2 to double the sending rate while delivery rate is increasing. This discovers *BtlBw* in log_2BDP RTTs but creates up to *2BDP* excess queue in the process. Once Startup finds *BtlBw*, BBR transitions to Drain, which uses the inverse of Startup's gain to get rid of the excess queue, then to ProbeBW once the *inflight* drops to a BDP.

Figure 6 shows the first second of a 10Mbps, 40ms BBR flow. The time/sequence plot shows the sender (green) and receiver (blue) progress vs. time. The red line shows a CUBIC sender under identical conditions. Vertical gray lines mark BBR state transitions. The lower figure shows the RTT of the two connections vs. time. Note that the time reference for this data is ack arrival (blue) so, while they appear to be time shifted, events are shown at the point where BBR learns of them and acts.

The lower graph of Figure 6 contrasts BBR and CUBIC. Their initial behavior is similar, but BBR completely drains its startup queue while CUBIC can't. Without a path model to tell it how much of the *inflight* is excess, CUBIC makes *inflight* growth less aggressive, but growth continues until either the bottleneck buffer fills and drops a packet or the receiver's *inflight* limit (TCP's receive window) is reached.

Figure 7 shows RTT behavior during the first eight seconds of the connections shown in Figure 6. CUBIC (red) fills the available buffer, then cycles from 70% to 100% full every few seconds. After startup, BBR (green) runs with essentially no queue.

Behavior of Multiple BBR Flows Sharing a Bottleneck

Figure 8 shows how individual throughputs for several BBR flows sharing a 100Mbps/10ms bottleneck converge to a fair share. The downward facing triangular structures are connection ProbeRTT states whose self-synchronization accelerates final convergence.

ProbeBW gain cycling (Figure 4) causes bigger flows to yield bandwidth to smaller flows, resulting in each learning its fair share. This happens fairly quickly (a few ProbeBW cycles), though unfairness can persist when late starters overestimate *RTprop* as a result of starting when other flows have (temporarily) created a queue.

To learn the true *RTProp*, a flow moves to the left of BDP using ProbeRTT state: when the *RTProp* estimate has not been updated (that is, by measuring a lower RTT) for many seconds, BBR enters ProbeRTT, which reduces the *inflight* to four packets for at least one round trip,

Figure 5. Bandwidth change.

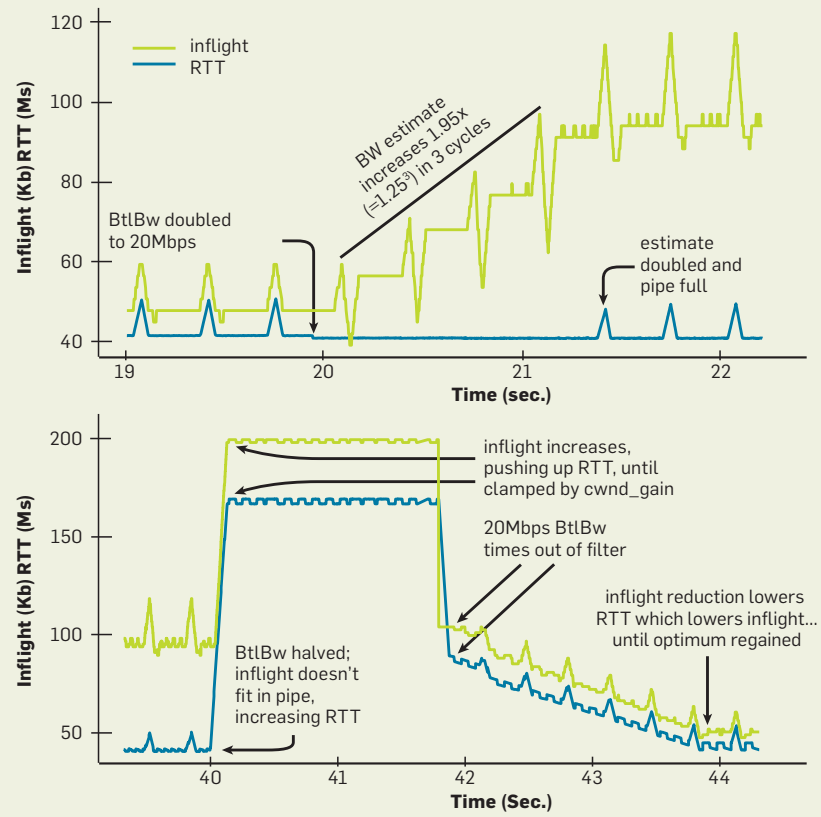
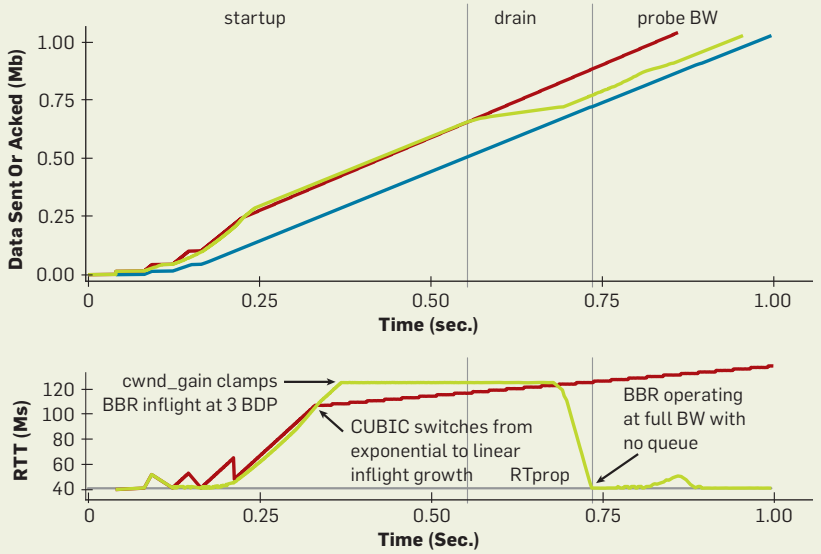


Figure 6. First second of a 10Mbps, 40ms BBR flow



then returns to the previous state. Large flows entering ProbeRTT drain many packets from the queue, so several flows see a new *RTprop* (new minimum RTT). This makes their *RTprop* estimates expire at the same time, so they enter ProbeRTT together, which makes the total queue dip larger and causes more flows to see a new *RTprop*, and so on. This distributed coordination is the key to both fairness and stability.

BBR synchronizes flows around the desirable event of an empty bottleneck queue. By contrast, loss-based congestion control synchronizes around the undesirable events of periodic queue growth and overflow, amplifying delay and packet loss.

Figure 7. First eight seconds of 10Mbps, 40ms cubic and BBR flows.

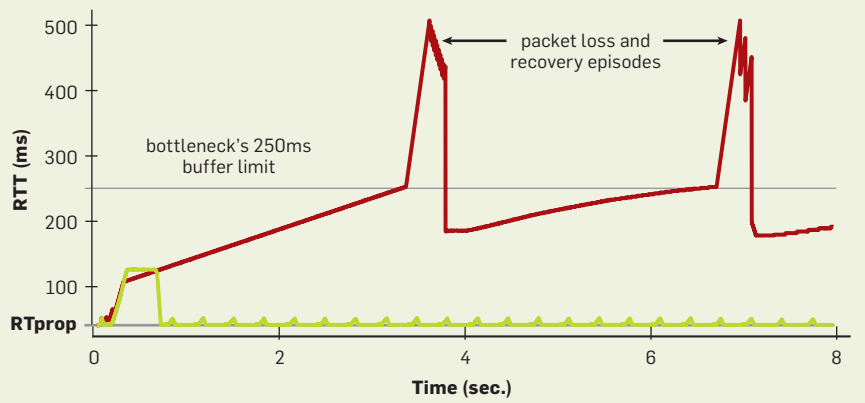


Figure 8. Throughputs of five BBR flows sharing a bottleneck

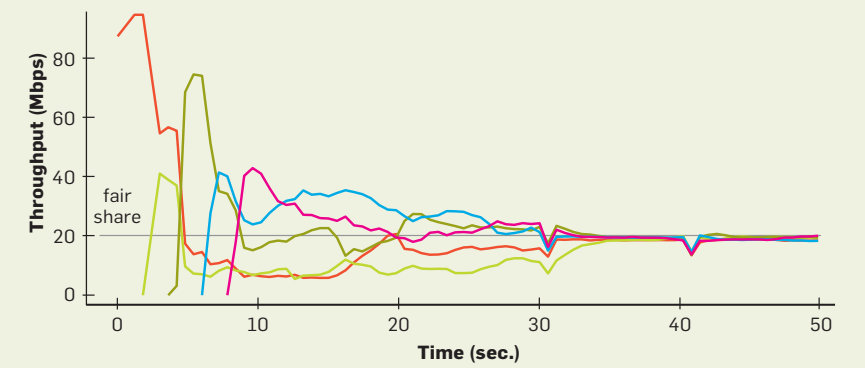
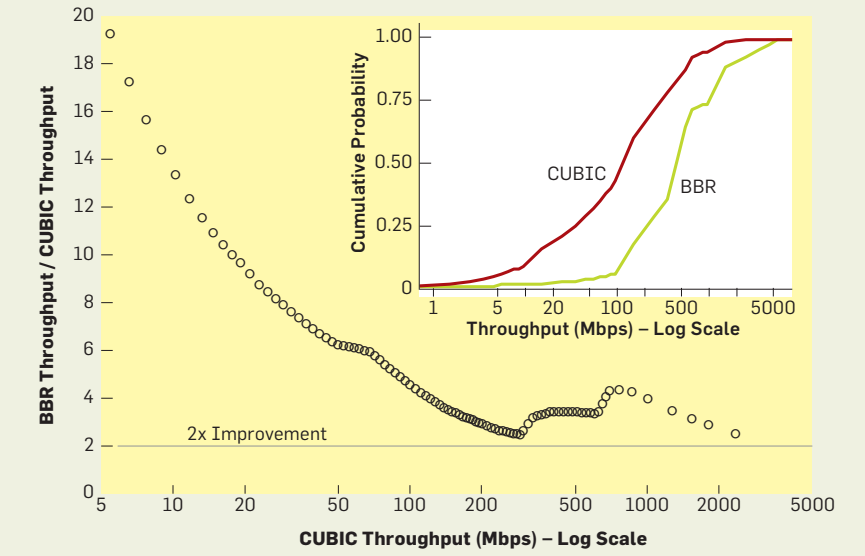


Figure 9. BBR vs. CUBIC relative throughput improvement.



Google B4 WAN Deployment Experience

Google's B4 network is a high-speed WAN (wide-area network) built using commodity switches.[15] Losses on these shallow-buffered switches result mostly from coincident arrivals of small traffic bursts. In 2015, Google started switching B4 production traffic from CUBIC to BBR. No issues or regressions were experienced, and since 2016 all B4 TCP traffic uses BBR. Figure 9 shows one reason for switching: BBR's throughput is consistently 2 to 25 times greater than CUBIC's. We had expected even more improvement but discovered that 75% of BBR connections were limited by the kernel's TCP receive buffer, which the network operations team had deliberately set low (8MB) to prevent CUBIC flooding the network with megabytes of excess *inflight* (8MB/200ms intercontinental RTT ⇒ 335Mbps max throughput). Manually raising the receive buffer on one U.S.-Europe path caused BBR immediately to reach 2Gbps, while CUBIC remained at 15Mbps—the 133x relative improvement predicted by Mathis et al.[17]

Figure 9 shows BBR vs. CUBIC relative throughput improvement; the inset shows throughput CDFs (cumulative distribution functions). Measures are from an active prober service that opens persistent BBR and CUBIC connections to remote datacenters, then transfers 8MB of data every minute. Probers communicate via many B4 paths within and between North America, Europe, and Asia.

The huge improvement is a direct consequence of BBR *not* using loss as a congestion indicator. To achieve full bandwidth, existing loss-based congestion controls require the loss rate to be less than the inverse square of the BDP[17] (for example, < one loss per 30 million packets for a 10Gbps/100ms path). Figure 10 compares measured goodput at various

loss rates. CUBIC's loss tolerance is a structural property of the algorithm, while BBR's is a configuration parameter. As BBR's loss rate approaches the ProbeBW peak gain, the probability of measuring a delivery rate of the true *BtlBw* drops sharply, causing the max filter to underestimate.

Figure 10 shows BBR vs. CUBIC goodput for 60-second flows on a 100Mbps/100ms link with 0.001 to 50% random loss. CUBIC's throughput decreases by 10 times at 0.1% loss and totally stalls above 1%. The maximum possible throughput is the link rate times fraction delivered (= *1 – lossRate*). BBR meets this limit up to a 5% loss and is close up to 15%.

YouTube Edge Deployment Experience

BBR is being deployed on Google.com and YouTube video servers. Google is running small-scale experiments in which a small percentage of users are randomly assigned either BBR or CUBIC. Playbacks using BBR show significant improvement in all of YouTube's quality-of-experience metrics, possibly because BBR's behavior is more consistent and predictable. BBR only slightly improves connection throughput because YouTube already adapts the server's streaming rate to well below *BtlBw* to minimize bufferbloat and rebuffer events. Even so, BBR reduces median RTT by 53% on average globally and by more than 80% in the developing world. Figure 11 shows BBR vs. CUBIC median RTT improvement from more than 200 million YouTube playback connections measured on five continents over a week.

More than half of the world's seven billion mobile Internet subscriptions connect via 8kbps to 114kbps 2.5G systems,[5] which suffer well-documented problems because of loss-based congestion control's buffer-filling propensities.[3] The bottleneck link for these systems is usually between the SGSN (serving GPRS support node)[18] and mobile device. SGSN software runs on a standard PC platform with ample memory, so there are frequently megabytes of buffer between the Internet and mobile device. Figure 12 compares (emulated) SGSN Internet-to-mobile delay for BBR and CUBIC.

Figure 10. BBR vs. CUBIC goodput under loss.

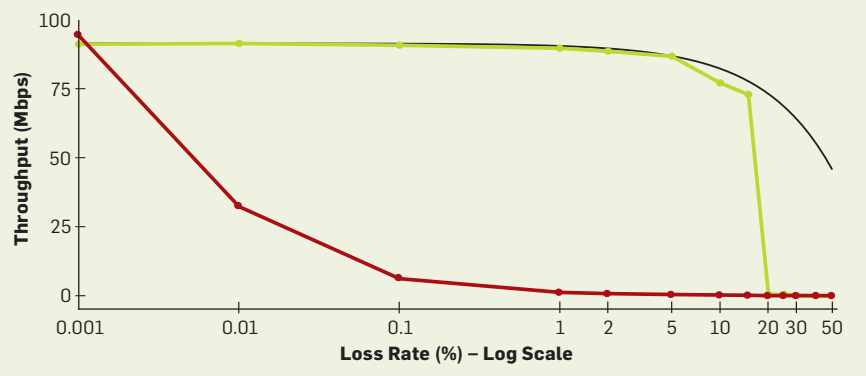


Figure 11. BBR vs. CUBIC median RTT improvement.

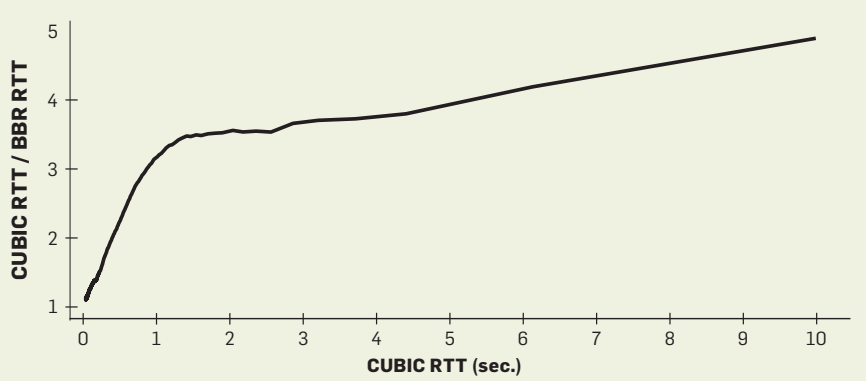
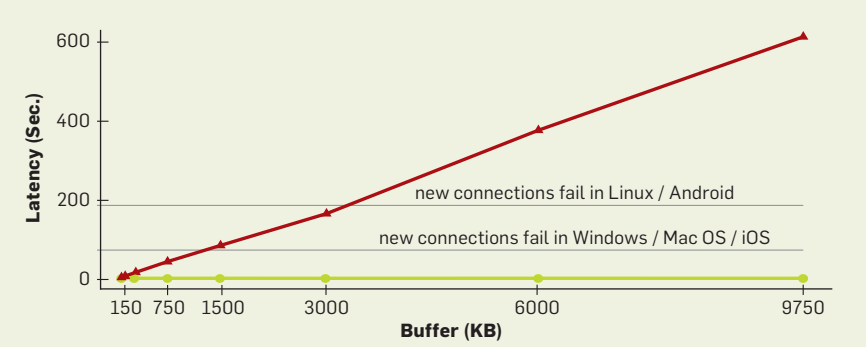


Figure 12. Steady-state median RTT variation with link buffer size



The horizontal lines mark one of the more serious consequences: TCP adapts to long RTT delay except on the connection initiation SYN packet, which has an OS-dependent fixed timeout. When the mobile device is receiving bulk data (for example, from automatic app updates) via a large-buffered SGSN, the device cannot connect to anything on the Internet until the queue empties (the SYN ACK accept packet is delayed for longer than the fixed SYN timeout).

Figure 12 shows steady-state median RTT variation with link buffer size on a 128Kbps/40ms link with eight BBR (green) or CUBIC (red) flows. BBR keeps the queue near its minimum, independent of both bottleneck buffer size and number of active flows. CUBIC flows always fill the buffer, so the delay grows linearly with buffer size.

Mobile Cellular Adaptive Bandwidth

Cellular systems adapt per-subscriber bandwidth based partly on a demand estimate that uses the queue of packets destined for the subscriber. Early versions of BBR were tuned to create very small queues, resulting in connections getting stuck at low rates. Raising the peak ProbeBW pacing_gain to create bigger queues resulted in fewer stuck connections, indicating it is possible to be too nice to some networks. With the current $1.25 \times BtlBw$ peak gain, no degradation is apparent compared with CUBIC on any network.

Delayed and stretched aks. Cellular, Wi-Fi, and cable broadband networks often delay and aggregate ACKs.[1] When *inflight* is limited to one BDP, this results in throughput-reducing stalls. Raising ProbeBW's cwnd_gain to two allowed BBR to continue sending smoothly at the estimated delivery rate, even when ACKs are delayed by up to one RTT. This largely avoids stalls.

Token-bucket policers. BBR's initial YouTube deployment revealed that most of the world's ISPs mangle traffic with token-bucket policers.[7] The bucket is typically full at connection startup so BBR learns the underlying network's *BtlBw*, but once the bucket empties, all packets sent faster than the (much lower than *BtlBw*) bucket fill rate are dropped. BBR eventually learns this new delivery rate, but the ProbeBW gain cycle results in continuous moderate losses. To minimize the upstream bandwidth waste and application latency increase from these losses, we added policer detection and an explicit policer model to BBR. We are also actively researching better ways to mitigate the policer damage.

Competition with loss-based congestion control. BBR converges toward a fair share of the bottleneck bandwidth whether competing with other BBR flows or with loss-based congestion control. Even as loss-based congestion control fills the available buffer, ProbeBW still robustly moves the *BtlBw* estimate toward the flow's fair share, and ProbeRTT finds an *RTProp* estimate just high enough for tit-for-tat convergence to a fair share. Unmanaged router buffers exceeding several BDPs, however, cause long-lived loss-based competitors to bloat the queue and grab more than their fair share. Mitigating this is another area of active research.

Conclusion

Rethinking congestion control pays big dividends. Rather than using events such as loss or buffer occupancy, which are only weakly correlated with congestion, BBR starts from Kleinrock's formal model of congestion and its associated optimal operating point. A pesky "impossibility" result that the crucial parameters of delay and bandwidth cannot be determined simultaneously is sidestepped by observing they can be estimated sequentially. Recent advances in control and estimation theory are then used to create a simple distributed control loop that verges on the optimum, fully utilizing the network while maintaining a small queue. Google's BBR implementation is available in the open source Linux kernel TCP.

BBR is deployed on Google's B4 backbone, improving throughput by orders of magnitude compared with CUBIC. It is also being deployed on Google and YouTube Web servers, substantially reducing latency on all five continents tested to date, most dramatically in developing regions. BBR runs purely on the sender and does not require changes to the protocol, receiver, or network, making it incrementally deployable. It depends only on RTT and packet-delivery acknowledgment, so can be implemented for most Internet transport protocols.

Acknowledgments

Thanks to Len Kleinrock for pointing out the right way to do congestion control and Larry Brakmo for pioneering work on Vegas[2] and New Vegas congestion control that presaged many elements of BBR, and for advice and guidance during BBR's early development. We also thank Eric Dumazet, Nandita Dukkipati, Jana Iyengar, Ian Swett, M. Fitz Nowlan, David Wetherall, Leonidas Kontothanassis, Amin Vahdat, and the Google BwE and YouTube infrastructure teams.

Related articles on queue.acm.org

Sender-side Buffers and the Case for Multimedia Adaptation
Aiman Erbad and Charles "Buck" Krasic
http://queue.acm.org/detail.cfm?id=2381998

You Don't Know Jack about Network Performance
Kevin Fall and Steve McCanne
http://queue.acm.org/detail.cfm?id=1066069

A Guided Tour through Data-center Networking
Dennis Abts and Bob Felderman
http://quue.acm.org/detail.cfm?id=2208919

References

1. Abrahamsson, M. TCP ACK suppression. IETF AQM mailing list, 2015; https://www.ietf.org/mail-archive/web/aqm/current/msg01480.html.
2. Brakmo, L.S., Peterson, L.L. TCP Vegas: End-to-end congestion avoidance on a global Internet. *IEEE J. Selected Areas in Communications 13*, 8 (1995), 1465–1480.
3. Chakravorty, R., Cartwright, J., Pratt, I. Practical experience with TCP over GPRS. *IEEE GLOBECOM*, 2002.
4. Corbet, J. TSO sizing and the FQ scheduler. LWN.net, 2013; https://lwn.net/Articles/564978/.
5. Ericsson. Ericsson Mobility Report (June), 2015; https://www.ericsson.com/res/docs/2015/ericsson-mobility-report-june-2015.pdf.
6. ESnet. Application tuning to optimize international astronomy workflow from NERSC to LFI-DPC at INAF-OATs; http://fasterdata.es.net/data-transfer-tools/case-studies/nersc-astronomy/.
7. Flach, T. et al. An Internet-wide analysis of traffic policing. *SIGCOMM*, 2016, 468–482.
8. Gail, R., Kleinrock, L. An invariant property of computer network power. In *Proceedings of the International Conference on Communications 63*, 1 (1981),1-63.1.5.
9. Gettys, J., Nichols, K. Bufferbloat: Dark buffers in the Internet. *acmqueue 9*, 11 (2011); http://queue.acm.org/detail.cfm?id=2071893.
10. Ha, S., Rhee, I. 2011. Taming the elephants: new TCP slow start. *Computer Networks 55*, 9 (2011), 2092–2110.
11. Ha, S., Rhee, I., Xu, L. CUBIC: a new TCP-friendly high-speed TCP variant. *ACM SIGOPS Operating Systems Review 42*, 5 (2008), 64–74.
12. Heidergott, B., Olsder, G. J., Van Der Woude, J. *Max Plus at Work: Modeling and Analysis of Synchronized Systems: a Course on Max-Plus Algebra and its Applications*. Princeton University Press, 2014.
13. Jacobson, V. Congestion avoidance and control. ACM *SIGCOMM Computer Communication Review 18*, 4 (1988): 314–329.
14. Jaffe, J. Flow control power is nondecentralizable. *IEEE Transactions on Communications 29*, 9 (1981), 1301–1306.
15. Jain, S. et al. B4: experience with a globally-deployed software defined WAN. *ACM SIGCOMM Computer Communication Review 43*, 4 (2013), 3–14.
16. Kleinrock, L. 1979. Power and deterministic rules of thumb for probabilistic problems in computer communications. In *Proceedings of the International Conference on Communications* (1979), 43.1.1-43.1.10.
17. Mathis, M., Semke, J., Mahdavi, J., Ott, T. The macroscopic behavior of the TCP congestion avoidance algorithm. *ACM SIGCOMM Computer Communication Review 27*, 3 (1997), 67–82.
18. Wikipedia. GPRS core network serving GPRS support node; https://en.wikipedia.org/wiki/GPRS_core_network#Serving_GPRS_support_node_.28SGSN.29.

Neal Cardwell, **Yuchung Cheng**, **C. Stephen Gunn**, **Soheil Hassas Yeganeh**, and **Van Jacobson** are members of Google's make-tcp-fast project, whose goal is to evolve Internet transport via fundamental research and open source software. Project contributions include TFO (TCP Fast Open), TLP (Tail Loss Probe), RACK loss recovery, fq/pacing, and a large fraction of the git commits to the Linux kernel TCP code for the past five years. They can be contacted at https://googlegroups.com/d/forum/bbr-dev.

Copyright held by owners/authors.